

TRƯỜNG ĐẠI HỌC KHOA HỌC HUẾ
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO TIỂU LUẬN
HỌC PHẦN: PHÁT TRIỂN ỨNG DỤNG IOT
MÃ HỌC PHẦN: TIN4024.004
ĐỀ TÀI: HỆ THỐNG BÁO CHÁY VỚI ESP32

Giảng viên hướng dẫn : Võ Việt Dũng

Sinh viên thực hiện : Hoàng Anh Quân – 22T1020354

Huế, 2026

MỤC LỤC

LỜI MỞ ĐẦU.....	1
CHƯƠNG 1: TỔNG QUAN VỀ HỆ THỐNG VÀ CÔNG NGHỆ SỬ DỤNG.	2
1.1. Tổng quan về hệ thống báo cháy	2
1.2. Vi điều khiển ESP32	3
1.3. Giao thức MQTT trong IoT	4
CHƯƠNG 2: THIẾT KẾ PHẦN CỨNG HỆ THỐNG.....	5
2.1. Mô hình tổng thể hệ thống	5
2.2. Lựa chọn cảm biến	5
2.2.1. Cảm biến khí MQ2	5
2.2.2. Cảm biến nhiệt độ DHT22	6
2.3. Khối xử lý trung tâm ESP32	6
2.4. Thiết bị cảnh báo và phụ trợ	7
2.5. Sơ đồ kết nối phần cứng	8
CHƯƠNG 3: THIẾT KẾ PHẦN MỀM VÀ THUẬT TOÁN.....	9
3.1. Nguyên lý hoạt động	9
3.2. Môi trường phát triển và mô phỏng.....	9
3.3. Xử lý dữ liệu và giao tiếp hệ thống	10
CHƯƠNG 4: TRIỂN KHAI MÔ PHỎNG VÀ ĐÁNH GIÁ HỆ THỐNG	10
4.1. Giới thiệu nền tảng Wokwi	10
4.2. Thiết kế và lập trình hệ thống.....	11
4.3. Cấu hình MQTT và nhận cảnh báo qua Telegram	11
4.4. Thực hành mô phỏng với các trường hợp	12
4.5. Tổng kết và đánh giá hệ thống.....	15
4.5.1. Hiệu quả hoạt động của hệ thống.....	15
4.5.2. Ưu điểm của hệ thống.....	16
4.5.3. Hạn chế và nhược điểm	16
KẾT LUẬN	17

LỜI MỞ ĐẦU

Trong những năm gần đây, cùng với sự phát triển nhanh chóng của đô thị hóa và công nghiệp hóa, nguy cơ xảy ra các sự cố cháy nổ ngày càng gia tăng và trở thành một vấn đề đáng báo động. Các vụ cháy không chỉ gây thiệt hại nghiêm trọng về tài sản mà còn đe dọa trực tiếp đến tính mạng con người. Đặc biệt, tại các khu dân cư đông đúc, nhà xưởng, phòng thí nghiệm hoặc các khu vực chứa vật liệu dễ cháy, việc phát hiện sớm các dấu hiệu cháy là yếu tố then chốt giúp giảm thiểu hậu quả.

Truyền thống, các hệ thống báo cháy thường hoạt động độc lập và chỉ phát tín hiệu cảnh báo tại chỗ như còi hú hoặc đèn báo. Tuy nhiên, các hệ thống này còn nhiều hạn chế, đặc biệt là trong trường hợp không có người trực tiếp tại hiện trường để kịp thời xử lý. Với sự phát triển của công nghệ Internet of Things (IoT), các hệ thống cảnh báo ngày nay có thể được nâng cấp để hoạt động thông minh hơn, cho phép giám sát từ xa và gửi cảnh báo theo thời gian thực đến người dùng thông qua mạng Internet.

Xuất phát từ nhu cầu thực tiễn đó, đề tài “Xây dựng hệ thống báo cháy sử dụng ESP32 kết hợp cảm biến và giao thức MQTT (Message Queuing Telemetry Transport)” được lựa chọn nhằm nghiên cứu và triển khai một mô hình cảnh báo cháy thông minh. Hệ thống sử dụng vi điều khiển ESP32 – một nền tảng mạnh mẽ, tích hợp sẵn kết nối Wi-Fi, kết hợp với cảm biến khí MQ2 để phát hiện khói và khí gas, cùng cảm biến nhiệt độ DHT22 để theo dõi sự thay đổi nhiệt độ môi trường. Khi phát hiện các dấu hiệu bất thường vượt ngưỡng cho phép, hệ thống sẽ kích hoạt cảnh báo tại chỗ thông qua buzzer, đồng thời gửi thông báo từ xa thông qua giao thức MQTT.

Giao thức MQTT được lựa chọn trong đề tài vì đây là một giao thức truyền thông nhẹ, tiêu tốn ít băng thông, phù hợp với các thiết bị IoT có tài nguyên hạn chế. Việc áp dụng MQTT giúp hệ thống có thể truyền dữ liệu nhanh chóng, ổn định và dễ dàng mở rộng trong tương lai.

Mục tiêu của đề tài là xây dựng một hệ thống báo cháy đơn giản, chi phí thấp nhưng vẫn đảm bảo hiệu quả trong việc phát hiện và cảnh báo sớm nguy cơ cháy. Đồng thời, đề tài cũng hướng tới việc làm quen với quy trình thiết kế một hệ thống IoT hoàn chỉnh, từ phần cứng, phần mềm đến giao thức truyền thông.

Thông qua quá trình thực hiện đề tài, nhóm không chỉ củng cố kiến thức về lập trình vi điều khiển, xử lý tín hiệu cảm biến mà còn hiểu rõ hơn về cách triển khai các hệ thống IoT trong thực tế. Đây sẽ là nền tảng quan trọng để phát triển các ứng dụng thông minh hơn trong tương lai.

CHƯƠNG 1: TỔNG QUAN VỀ HỆ THỐNG VÀ CÔNG NGHỆ SỬ DỤNG.

1.1. Tổng quan về hệ thống báo cháy

Hệ thống báo cháy là một tập hợp các thiết bị điện tử được thiết kế nhằm phát hiện sớm các dấu hiệu của đám cháy như khói, nhiệt độ tăng cao hoặc sự xuất hiện của khí dễ cháy trong môi trường. Khi phát hiện các dấu hiệu bất thường, hệ thống sẽ phát ra tín hiệu cảnh báo để con người có thể kịp thời xử lý, từ đó giảm thiểu thiệt hại về người và tài sản.

Một hệ thống báo cháy cơ bản thường bao gồm các thành phần chính sau:

- **Thiết bị cảm biến (Sensor):** Có nhiệm vụ phát hiện các yếu tố liên quan đến cháy như khói, nhiệt độ hoặc khí gas. Đây là thành phần quan trọng nhất, quyết định độ chính xác và độ nhạy của hệ thống.
- **Bộ xử lý trung tâm:** Tiếp nhận tín hiệu từ các cảm biến, phân tích và đưa ra quyết định có kích hoạt cảnh báo hay không. Trong các hệ thống hiện đại, vi điều khiển như ESP32 thường được sử dụng để đảm nhận vai trò này.
- **Thiết bị cảnh báo:** Bao gồm còi (buzzer), đèn LED hoặc các phương tiện thông báo khác, giúp cảnh báo trực tiếp tại hiện trường.
- **Hệ thống truyền thông:** Cho phép gửi thông tin cảnh báo đến các thiết bị từ xa thông qua Internet hoặc mạng nội bộ, đặc biệt quan trọng trong các hệ thống IoT.

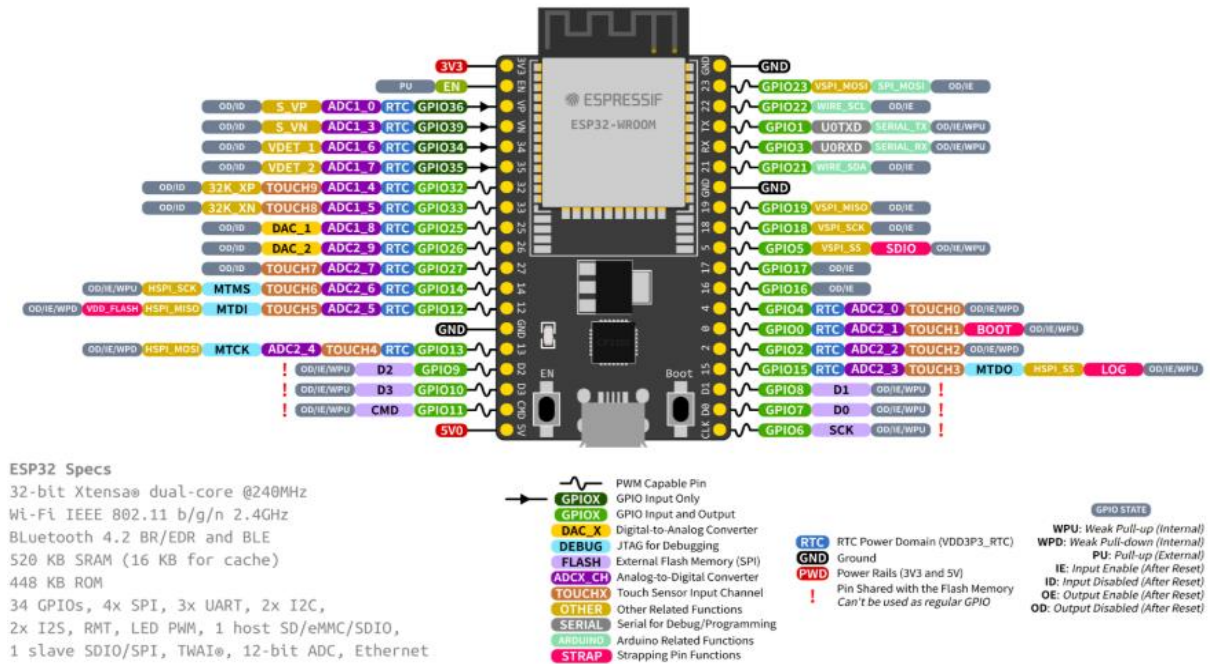
Dựa vào nguyên lý hoạt động, hệ thống báo cháy có thể được chia thành hai loại chính:

- **Hệ thống báo cháy truyền thống:** Hoạt động độc lập, chỉ phát tín hiệu cảnh báo tại chỗ. Loại này đơn giản, dễ lắp đặt nhưng hạn chế về khả năng giám sát từ xa.
- **Hệ thống báo cháy thông minh (IoT):** Kết nối với Internet, cho phép giám sát và điều khiển từ xa. Các hệ thống này có khả năng gửi cảnh báo qua điện thoại, máy tính hoặc các nền tảng trực tuyến, nâng cao hiệu quả sử dụng.

Trong bối cảnh hiện nay, các hệ thống báo cháy thông minh ngày càng được ưa chuộng do khả năng hoạt động linh hoạt, chi phí hợp lý và dễ dàng mở rộng. Việc tích hợp các cảm biến với vi điều khiển và giao thức truyền thông như MQTT giúp hệ thống có thể hoạt động ổn định, truyền dữ liệu nhanh chóng và đáp ứng tốt các yêu cầu thực tế.

Nhìn chung, hệ thống báo cháy đóng vai trò quan trọng trong việc đảm bảo an toàn phòng chống cháy nổ, đặc biệt trong các môi trường có nguy cơ cao như nhà ở, nhà kho, xưởng sản xuất và các khu vực công cộng. Việc nghiên cứu và phát triển các hệ thống báo cháy thông minh là xu hướng tất yếu trong thời đại công nghệ số hiện nay.

1.2. Vi điều khiển ESP32



ESP32 là một vi điều khiển do hãng Espressif Systems phát triển, được sử dụng rộng rãi trong các ứng dụng Internet of Things (IoT) nhờ tích hợp sẵn các tính năng kết nối không dây như Wi-Fi và Bluetooth. Đây là một trong những nền tảng phần cứng phổ biến, phù hợp cho các hệ thống nhúng yêu cầu khả năng xử lý tốt và giao tiếp linh hoạt.

Về cấu trúc, ESP32 được trang bị bộ vi xử lý lõi kép (dual-core) 32-bit, hoạt động với xung nhịp lên đến 240 MHz, cho phép xử lý dữ liệu nhanh và hiệu quả. Ngoài ra, vi điều khiển này còn tích hợp bộ nhớ RAM và Flash đủ lớn để phục vụ cho các ứng dụng phức tạp. ESP32 hỗ trợ nhiều chuẩn giao tiếp như GPIO, ADC (Analog to Digital Converter), DAC, UART, SPI, I2C, giúp dễ dàng kết nối với các loại cảm biến và thiết bị ngoại vi.

Một trong những ưu điểm nổi bật của ESP32 là khả năng kết nối Wi-Fi mạnh mẽ, cho phép thiết bị truy cập Internet và truyền dữ liệu theo thời gian thực. Bên cạnh đó, Bluetooth (Classic và BLE) cũng được tích hợp, mở rộng khả năng giao tiếp với các thiết bị di động hoặc hệ thống khác.

Trong đề tài hệ thống báo cháy, ESP32 đóng vai trò là bộ xử lý trung tâm với các chức năng chính:

- Thu thập dữ liệu từ các cảm biến như MQ2 và DHT22
- Xử lý và phân tích dữ liệu để phát hiện nguy cơ cháy
- Điều khiển các thiết bị cảnh báo như buzzer
- Gửi dữ liệu và cảnh báo đến máy chủ thông qua giao thức MQTT

Ngoài ra, ESP32 còn có ưu điểm về tiêu thụ năng lượng thấp, phù hợp với các hệ thống hoạt động liên tục trong thời gian dài. Giá thành hợp lý và cộng đồng hỗ trợ lớn cũng là những yếu tố giúp ESP32 trở thành lựa chọn hàng đầu trong các dự án IoT.

1.3. Giao thức MQTT trong IoT

MQTT (Message Queuing Telemetry Transport) là một giao thức truyền thông nhẹ, được thiết kế đặc biệt cho các hệ thống có băng thông hạn chế, độ trễ cao hoặc tài nguyên phần cứng hạn chế như trong các ứng dụng Internet of Things (IoT). Giao thức này hoạt động dựa trên mô hình publish/subscribe, giúp việc trao đổi dữ liệu giữa các thiết bị trở nên linh hoạt và hiệu quả hơn so với mô hình client-server truyền thống.

Trong mô hình MQTT, hệ thống bao gồm ba thành phần chính:

- **Publisher (bên gửi):** Là các thiết bị thu thập dữ liệu và gửi thông tin lên hệ thống. Trong đề tài này, ESP32 đóng vai trò là Publisher, gửi dữ liệu cảm biến và cảnh báo cháy.
- **Broker (máy chủ trung gian):** Là trung tâm xử lý và phân phối dữ liệu. Broker nhận dữ liệu từ Publisher và chuyển tiếp đến các Subscriber phù hợp.
- **Subscriber (bên nhận):** Là các thiết bị hoặc ứng dụng đăng ký nhận dữ liệu từ Broker theo các chủ đề (topic) nhất định, ví dụ như ứng dụng điện thoại hoặc web giám sát.

Một khái niệm quan trọng trong MQTT là Topic. Topic là kênh dữ liệu mà Publisher gửi thông tin và Subscriber đăng ký để nhận dữ liệu. Ví dụ, trong hệ thống báo cháy, một topic có thể được đặt là: fire/alert

Khi ESP32 phát hiện nguy cơ cháy, nó sẽ gửi thông báo lên topic này và các thiết bị đã đăng ký sẽ nhận được cảnh báo ngay lập tức.

MQTT cung cấp ba mức độ đảm bảo chất lượng dịch vụ (QoS – Quality of Service):

- QoS 0: Gửi dữ liệu tối đa một lần (không đảm bảo nhận)
- QoS 1: Đảm bảo gửi ít nhất một lần
- QoS 2: Đảm bảo gửi đúng một lần (độ tin cậy cao nhất)

Ưu điểm nổi bật của MQTT trong các ứng dụng IoT bao gồm:

- Nhẹ và tiết kiệm băng thông: Gói tin nhỏ, phù hợp với mạng yếu
- Hiệu quả cao: Giảm tải cho thiết bị nhúng
- Dễ mở rộng: Nhiều thiết bị có thể tham gia hệ thống
- Truyền dữ liệu thời gian thực: Phù hợp cho các hệ thống giám sát

Trong đề tài hệ thống báo cháy, MQTT được sử dụng để gửi cảnh báo từ ESP32 đến các thiết bị giám sát từ xa. Khi phát hiện nhiệt độ cao hoặc nồng độ khí vượt ngưỡng, ESP32 sẽ publish thông điệp cảnh báo lên Broker. Ngay lập tức, các Subscriber như

ứng dụng điện thoại hoặc dashboard web sẽ nhận được thông báo, giúp người dùng phản ứng nhanh với tình huống nguy hiểm.

CHƯƠNG 2: THIẾT KẾ PHẦN CỨNG HỆ THỐNG

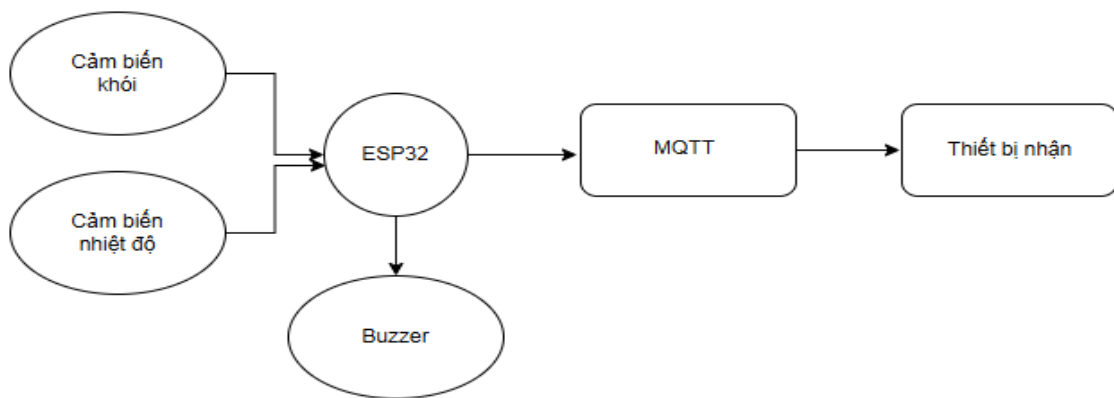
2.1. Mô hình tổng thể hệ thống

Hệ thống báo cháy được thiết kế gồm 4 khối chính:

- Khối cảm biến: Gồm MQ2 (phát hiện khói, khí gas) và DHT22 (đo nhiệt độ).
- Khối xử lý: ESP32 thu thập và xử lý dữ liệu từ cảm biến.
- Khối cảnh báo: Buzzer phát tín hiệu khi có nguy cơ cháy.
- Khối truyền thông: ESP32 gửi dữ liệu cảnh báo qua Wi-Fi sử dụng giao thức MQTT.

Nguyên lý hoạt động:

Cảm biến thu thập dữ liệu → ESP32 xử lý → nếu vượt ngưỡng thì kích hoạt buzzer và gửi cảnh báo qua MQTT, ngược lại tiếp tục giám sát.



2.2. Lựa chọn cảm biến

2.2.1. Cảm biến khí MQ2



Cảm biến MQ2 là loại cảm biến khí phổ biến, được sử dụng để phát hiện các loại khí dễ cháy như LPG, methane, propane và khói trong môi trường. Đây là cảm biến rất phù hợp trong các hệ thống cảnh báo cháy nhờ độ nhạy cao và khả năng phản hồi nhanh.

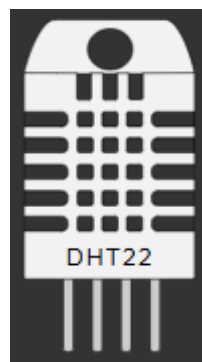
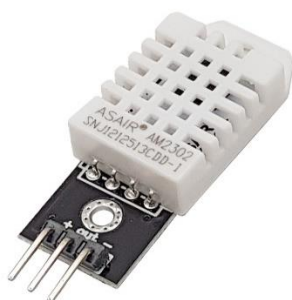
Về nguyên lý hoạt động, MQ2 sử dụng vật liệu bán dẫn (SnO_2). Khi có khí cháy xuất hiện, điện trở của cảm biến sẽ thay đổi, dẫn đến sự thay đổi điện áp đầu ra. ESP32 sẽ đọc giá trị này thông qua chân analog để xác định nồng độ khí.

Đặc điểm chính:

- Phát hiện được nhiều loại khí và khói
- Xuất tín hiệu analog (có thể đọc bằng ADC)
- Độ nhạy cao nhưng cần hiệu chỉnh ngưỡng
- Cần thời gian làm nóng (warm-up) trước khi sử dụng

Trong hệ thống báo cháy, MQ2 đóng vai trò phát hiện sớm sự xuất hiện của khói hoặc khí gas – một dấu hiệu quan trọng của cháy.

2.2.2. Cảm biến nhiệt độ DHT22



DHT22 là cảm biến dùng để đo nhiệt độ và độ ẩm với độ chính xác cao, thường được sử dụng trong các ứng dụng IoT.

Cảm biến hoạt động dựa trên cảm biến nhiệt điện trở và cảm biến độ ẩm điện dung, sau đó chuyển đổi dữ liệu sang dạng tín hiệu số để gửi về vi điều khiển.

Đặc điểm chính:

- Đo được cả nhiệt độ và độ ẩm
- Độ chính xác cao hơn DHT11
- Giao tiếp digital (1 dây) giúp dễ sử dụng
- Dải đo nhiệt độ rộng (khoảng -40°C đến 80°C)

Trong hệ thống báo cháy, DHT22 giúp theo dõi sự tăng nhiệt bất thường của môi trường. Khi nhiệt độ vượt ngưỡng cho phép, hệ thống sẽ xác định nguy cơ cháy và kích hoạt cảnh báo.

2.3. Khối xử lý trung tâm ESP32

Khối xử lý trung tâm sử dụng vi điều khiển ESP32, đóng vai trò là “bộ não” của toàn bộ hệ thống. ESP32 có nhiệm vụ thu thập dữ liệu từ các cảm biến, xử lý thông tin và điều khiển các hoạt động của hệ thống.

Trong hệ thống báo cháy, ESP32 thực hiện các chức năng chính sau:

- Thu thập dữ liệu:
Nhận tín hiệu từ cảm biến MQ2 (dạng analog) và DHT22 (dạng digital).
- Xử lý và phân tích:
Dữ liệu thu được sẽ được ESP32 xử lý và so sánh với các ngưỡng đã cài đặt (ví dụ: nồng độ khí hoặc nhiệt độ). Nếu vượt ngưỡng, hệ thống sẽ xác định có nguy cơ cháy.
- Điều khiển thiết bị:
Khi phát hiện nguy hiểm, ESP32 sẽ kích hoạt buzzer để cảnh báo tại chỗ.
- Truyền dữ liệu:
Sử dụng Wi-Fi tích hợp, ESP32 gửi thông báo cảnh báo lên MQTT Broker để các thiết bị khác có thể nhận được.

Ngoài ra, ESP32 còn hỗ trợ nhiều chuẩn giao tiếp như ADC, GPIO, UART, I2C, giúp dễ dàng kết nối với các cảm biến và thiết bị ngoại vi khác.

Nhờ khả năng xử lý mạnh, tích hợp Wi-Fi và tính linh hoạt cao, ESP32 là lựa chọn phù hợp để làm trung tâm điều khiển trong hệ thống báo cháy IoT.

2.4. Thiết bị cảnh báo và phụ trợ

Trong hệ thống báo cháy, ngoài các thành phần chính, các linh kiện phụ trợ đóng vai trò hỗ trợ cảnh báo và đảm bảo hệ thống hoạt động ổn định.

Buzzer (bz1) – thiết bị cảnh báo



Buzzer là thiết bị được sử dụng để phát tín hiệu âm thanh khi hệ thống phát hiện nguy cơ cháy. Đây là hình thức cảnh báo cục bộ giúp người dùng nhận biết ngay lập tức sự cố xảy ra.

Khi ESP32 xác định giá trị cảm biến vượt ngưỡng an toàn (khí gas hoặc nhiệt độ), buzzer sẽ được kích hoạt để phát ra âm thanh cảnh báo.

Kết nối Buzzer với ESP32

Dựa trên sơ đồ kết nối của hệ thống, buzzer (ID: bz1) được nối với ESP32 như sau:

- Chân 2 của buzzer → kết nối với GPIO15 của ESP32 (dây màu xanh lá)

- Chân 1 của buzzer → kết nối với GND của ESP32 (dây màu đen)

Khi GPIO15 được xuất tín hiệu HIGH, buzzer sẽ hoạt động và phát âm thanh.

Lưu ý khi sử dụng

Trong thực tế, để đảm bảo an toàn cho mạch điện và tăng độ bền cho thiết bị, cần:

- Sử dụng điện trở hạn dòng mắc nối tiếp với buzzer
- Đảm bảo nguồn cấp phù hợp với thông số của buzzer

Vai trò trong hệ thống

Buzzer giúp:

- Cảnh báo trực tiếp tại chỗ khi có nguy cơ cháy
- Tăng khả năng phản ứng nhanh của người dùng
- Kết hợp với MQTT để tạo hệ thống cảnh báo kép (tại chỗ + từ xa)

2.5. Sơ đồ kết nối phần cứng

Bảng dưới đây mô tả chi tiết các kết nối giữa các linh kiện và vi điều khiển ESP32 trong hệ thống:

Linh kiện	Chân linh kiện	ESP32	Chức năng	Màu dây
DHT22	SDA (DATA)	GPIO4	Truyền dữ liệu nhiệt độ và độ ẩm	Xanh lá
DHT22	VCC	3V3	Cấp nguồn cho DHT22	Đỏ
DHT22	GND	GND.2	Mass (Ground)	Đen
MQ2	AO	GPIO35	Tín hiệu analog nồng độ khí/khói	Xanh lá
MQ2	VCC	3V3	Cấp nguồn cho MQ2	Đỏ
MQ2	GND	GND.1	Mass (Ground)	Đen
Buzzer	Chân 2	GPIO15	Chân điều khiển buzzer (output)	Xanh lá
Buzzer	Chân 1	GND.1	Mass (Ground)	Đen
ESP32 DevKitC V4	TX	Serial RX	Gửi dữ liệu đến Serial Monitor	-
ESP32 DevKitC V4	RX	Serial TX	Nhận dữ liệu từ Serial Monitor	-

CHƯƠNG 3: THIẾT KẾ PHẦN MỀM VÀ THUẬT TOÁN

3.1. Nguyên lý hoạt động

Hệ thống báo cháy được thiết kế hoạt động theo cơ chế giám sát liên tục các thông số môi trường và chỉ phát cảnh báo khi phát hiện nguy cơ cháy. Khi được cấp nguồn, vi điều khiển ESP32 khởi động và bắt đầu thực hiện chương trình điều khiển.

Trước tiên, ESP32 tiến hành kết nối với mạng Wi-Fi đã được cấu hình sẵn. Quá trình này đóng vai trò quan trọng vì là điều kiện cần để hệ thống có thể giao tiếp với MQTT Broker. Trong trường hợp kết nối không thành công, ESP32 sẽ tự động thử lại cho đến khi thiết lập được kết nối ổn định. Sau khi kết nối Wi-Fi thành công, ESP32 tiếp tục thiết lập kết nối với MQTT Broker. Đây là bước cần thiết để hệ thống có thể gửi các thông báo cảnh báo đến các thiết bị giám sát từ xa.

Khi quá trình kết nối hoàn tất, ESP32 bắt đầu đọc dữ liệu từ các cảm biến. Cảm biến MQ2 cung cấp giá trị analog phản ánh nồng độ khói và khí trong môi trường, trong khi cảm biến DHT22 cung cấp giá trị nhiệt độ hiện tại. Các dữ liệu này được ESP32 xử lý và so sánh với các ngưỡng đã được thiết lập trước.

Cụ thể, nếu giá trị từ cảm biến MQ2 vượt quá ngưỡng tương ứng khoảng 300 (tương đương mức khói nguy hiểm), hệ thống sẽ xác định có nguy cơ cháy do khói. Khi đó, ESP32 sẽ kích hoạt buzzer để phát cảnh báo tại chỗ, đồng thời gửi thông báo qua MQTT với nội dung cảnh báo cháy.

Trong trường hợp giá trị MQ2 không vượt ngưỡng, hệ thống sẽ tiếp tục kiểm tra giá trị nhiệt độ. Nếu nhiệt độ đo được từ cảm biến DHT22 vượt quá 50°C, hệ thống sẽ xác định có nguy cơ cháy do nhiệt độ cao. Tương tự, buzzer sẽ được kích hoạt và một thông báo cảnh báo sẽ được gửi qua MQTT.

Ngược lại, nếu cả hai giá trị khói và nhiệt độ đều nằm trong ngưỡng an toàn, hệ thống sẽ không phát cảnh báo và buzzer sẽ ở trạng thái tắt. Sau mỗi chu kỳ kiểm tra, hệ thống sẽ tạm dừng trong một khoảng thời gian ngắn trước khi tiếp tục đọc dữ liệu mới, đảm bảo việc giám sát môi trường diễn ra liên tục và ổn định.

Như vậy, hệ thống chỉ phát cảnh báo khi thực sự phát hiện nguy cơ cháy, giúp giảm thiểu cảnh báo sai và nâng cao hiệu quả hoạt động.

3.2. Môi trường phát triển và mô phỏng

Trong đề tài này, hệ thống được phát triển và kiểm thử thông qua môi trường lập trình và mô phỏng trực tuyến. Vi điều khiển ESP32 được lập trình bằng ngôn ngữ C/C++ với các thư viện hỗ trợ như WiFi.h, PubSubClient và DHT để thực hiện kết nối mạng, giao tiếp MQTT và đọc dữ liệu cảm biến.

Bên cạnh đó, hệ thống được mô phỏng trên nền tảng Wokwi. Công cụ này cho phép thiết kế mạch, kết nối các linh kiện và chạy chương trình trực tiếp, giúp kiểm tra hoạt động của hệ thống một cách nhanh chóng.

Ngoài ra, hệ thống sử dụng MQTT Broker (kết hợp với Telegram) để gửi và nhận thông báo cảnh báo. Khi phát hiện nguy cơ cháy, ESP32 sẽ gửi thông điệp qua MQTT

và chuyển tiếp đến Telegram, giúp người dùng nhận được cảnh báo từ xa một cách kịp thời.

3.3. Xử lý dữ liệu và giao tiếp hệ thống

ESP32 được lập trình để đọc dữ liệu từ các cảm biến nhằm phát hiện nguy cơ cháy.

- Đọc cảm biến MQ2 (GPIO35):
ESP32 sử dụng hàm `analogRead()` để đọc giá trị từ chân GPIO35. Giá trị này phản ánh nồng độ khói/khí trong môi trường và được so sánh với ngưỡng đã thiết lập (≈ 300 tương ứng khoảng $> 6\%$ khói).
- Đọc cảm biến DHT22 (GPIO4):
ESP32 sử dụng thư viện DHT để đọc nhiệt độ từ cảm biến thông qua chân GPIO4. Giá trị nhiệt độ được sử dụng trực tiếp để đánh giá nguy cơ cháy.
- Xử lý dữ liệu:
Dữ liệu từ các cảm biến được ESP32 xử lý và so sánh với các ngưỡng:
 - Ngưỡng khói: $MQ2 > 300$
 - Ngưỡng nhiệt độ: $> 50^{\circ}\text{C}$

Khi một trong hai giá trị vượt ngưỡng, hệ thống xác định có nguy cơ cháy.

ESP32 được cấu hình để tự động kết nối với mạng Wi-Fi thông qua SSID và mật khẩu đã khai báo. Sau khi khởi động, thiết bị sẽ tiến hành kết nối và kiểm tra trạng thái liên tục.

Trong trường hợp mất kết nối, ESP32 sẽ tự động thử kết nối lại nhằm đảm bảo hệ thống luôn duy trì khả năng truyền dữ liệu.

Sau khi kết nối Wi-Fi thành công, ESP32 thiết lập kết nối với MQTT Broker (kết hợp với Telegram). ESP32 đóng vai trò là Publisher, gửi các thông điệp cảnh báo đến một topic xác định.

Khi phát hiện nguy cơ cháy ($MQ2 > 300$ hoặc nhiệt độ $> 50^{\circ}\text{C}$), ESP32 sẽ tạo thông điệp chứa thông tin cảnh báo (khói hoặc nhiệt độ) và gửi qua MQTT. Thông điệp này sau đó được chuyển tiếp đến Telegram để người dùng nhận cảnh báo từ xa.

CHƯƠNG 4: TRIỂN KHAI MÔ PHỎNG VÀ ĐÁNH GIÁ HỆ THỐNG

4.1. Giới thiệu nền tảng Wokwi

Wokwi là một nền tảng mô phỏng mạch điện tử trực tuyến, cho phép người dùng thiết kế, lập trình và kiểm thử các hệ thống nhúng mà không cần phần cứng thực tế. Nền tảng này hỗ trợ nhiều loại vi điều khiển phổ biến như ESP32, Arduino, cùng với các linh kiện như cảm biến, LED, buzzer,...

Với giao diện trực quan, Wokwi cho phép người dùng kéo thả linh kiện, kết nối mạch và viết chương trình ngay trên trình duyệt. Ngoài ra, hệ thống còn hỗ trợ mô phỏng hoạt động thực tế của cảm biến và hiển thị kết quả thông qua Serial Monitor.

4.2. Thiết kế và lập trình hệ thống

Trong quá trình thực hành, hệ thống được triển khai trên nền tảng Wokwi với các linh kiện chính gồm ESP32, cảm biến MQ2, cảm biến DHT22 và buzzer. Việc đấu nối các linh kiện được thực hiện như sau:

- Cảm biến MQ2:
Chân AO được nối với GPIO35 của ESP32 để đọc giá trị analog nồng độ khí/khói. Chân VCC nối 3V3 và GND nối mass.
- Cảm biến DHT22:
Chân DATA nối với GPIO4 của ESP32 để truyền dữ liệu nhiệt độ. Chân VCC nối 3V3 và GND nối mass.
- Buzzer:
Chân điều khiển (chân 2) nối với GPIO15 của ESP32, chân còn lại nối GND để hoàn thành mạch.

Sau khi hoàn tất đấu nối, chương trình được nạp vào ESP32 để thực hiện các chức năng chính.

Về lập trình, ESP32 sử dụng hàm `analogRead(GPIO35)` để đọc giá trị từ cảm biến MQ2. Giá trị này nằm trong khoảng từ 0 đến 4095 và được dùng để xác định nồng độ khói. Đồng thời, cảm biến DHT22 được đọc thông qua thư viện DHT với lệnh `dht.readTemperature()` để lấy giá trị nhiệt độ môi trường.

Dữ liệu từ hai cảm biến sau đó được xử lý và so sánh với ngưỡng đã thiết lập:

- $MQ2 > 300 \rightarrow$ có khói
- Nhiệt độ $> 50^{\circ}C \rightarrow$ nguy cơ cháy

Khi một trong hai điều kiện xảy ra, ESP32 sẽ xuất tín hiệu HIGH tại GPIO15 để kích hoạt buzzer phát âm thanh cảnh báo.

Ngoài ra, ESP32 còn gửi dữ liệu cảnh báo qua MQTT bằng cách sử dụng thư viện `PubSubClient`. Thông điệp cảnh báo sẽ được gửi lên topic với nội dung tương ứng.

Trong quá trình chạy, các giá trị cảm biến và trạng thái hệ thống được hiển thị trên Serial Monitor để người dùng dễ dàng theo dõi và kiểm tra.

Việc thực hành trên Wokwi cho phép thay đổi giá trị cảm biến trực tiếp để kiểm tra phản ứng của hệ thống, từ đó đánh giá độ chính xác và tính ổn định của chương trình.

4.3. Cấu hình MQTT và nhận cảnh báo qua Telegram

Hệ thống sử dụng ESP32 gửi cảnh báo khi phát hiện nguy cơ cháy qua MQTT, Telegram được dùng để nhận tin nhắn cảnh báo từ xa.

- Cấu hình MQTT: ESP32 kết nối với Broker và gửi dữ liệu lên topic.
- Gửi cảnh báo: Khi giá trị analog $MQ2 > 300$ (tương đương ~ 600 ppm khói) hoặc nhiệt độ $> 50^{\circ}C$, ESP32 gửi thông điệp “CHÁY - KHÓI” hoặc “CHÁY - NHIỆT ĐỘ”.

- Nhận cảnh báo qua Telegram: Một Telegram Bot đọc dữ liệu từ MQTT và gửi tin nhắn đến người dùng, thông báo ngay lập tức trên điện thoại.

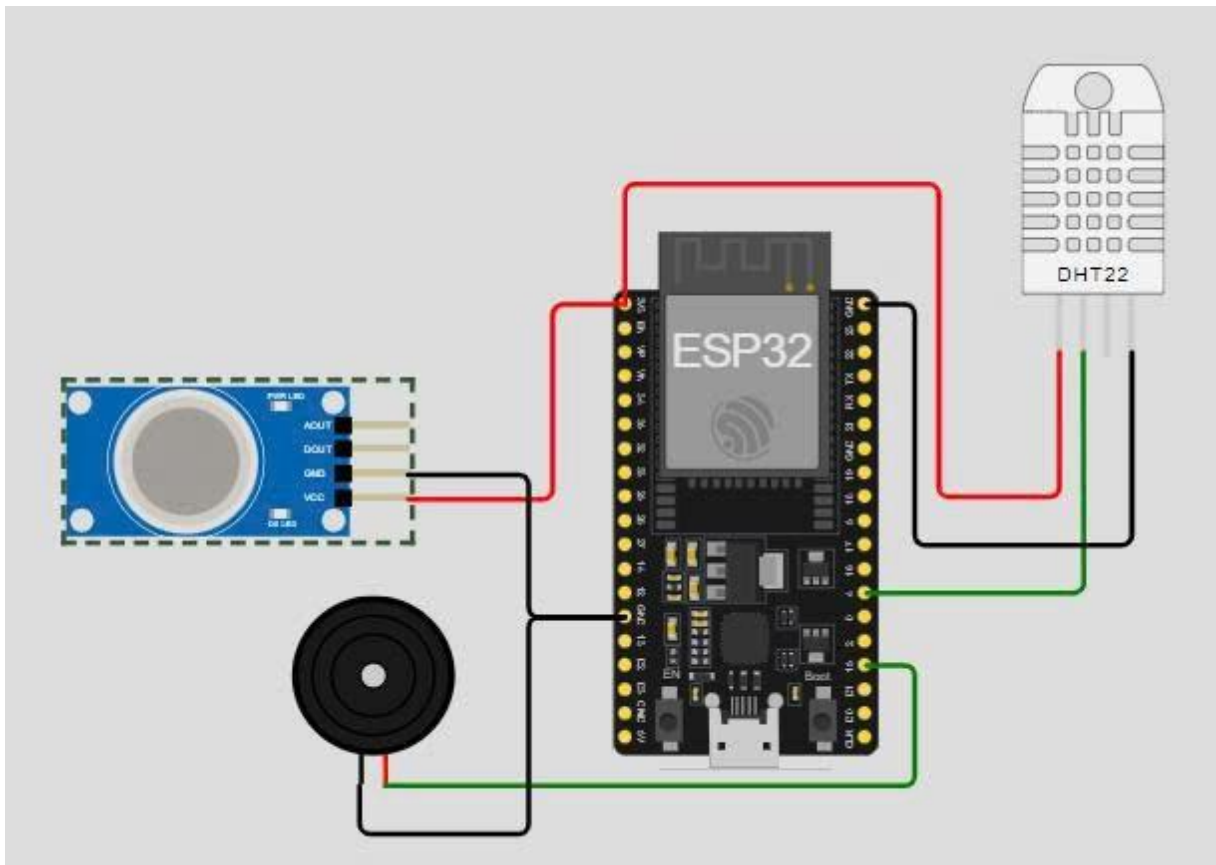
Hệ thống vừa cảnh báo bằng buzzer cục bộ, vừa gửi cảnh báo từ xa qua Telegram, giúp phát hiện cháy kịp thời và chính xác.

4.4. Thực hành mô phỏng với các trường hợp

Hệ thống được mô phỏng trên Wokwi để kiểm tra phản ứng của ESP32 với các giá trị cảm biến khác nhau.

Trường hợp 1: Nhiệt độ và khói chưa vượt ngưỡng

- Thiết lập mô phỏng:
 - MQ2 analog < 300 (khói < 6%)
 - DHT22 temperature < 50°C

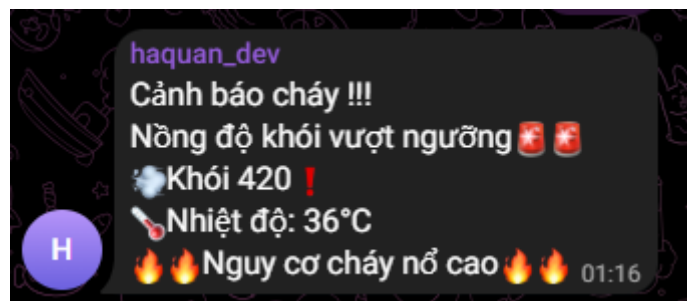
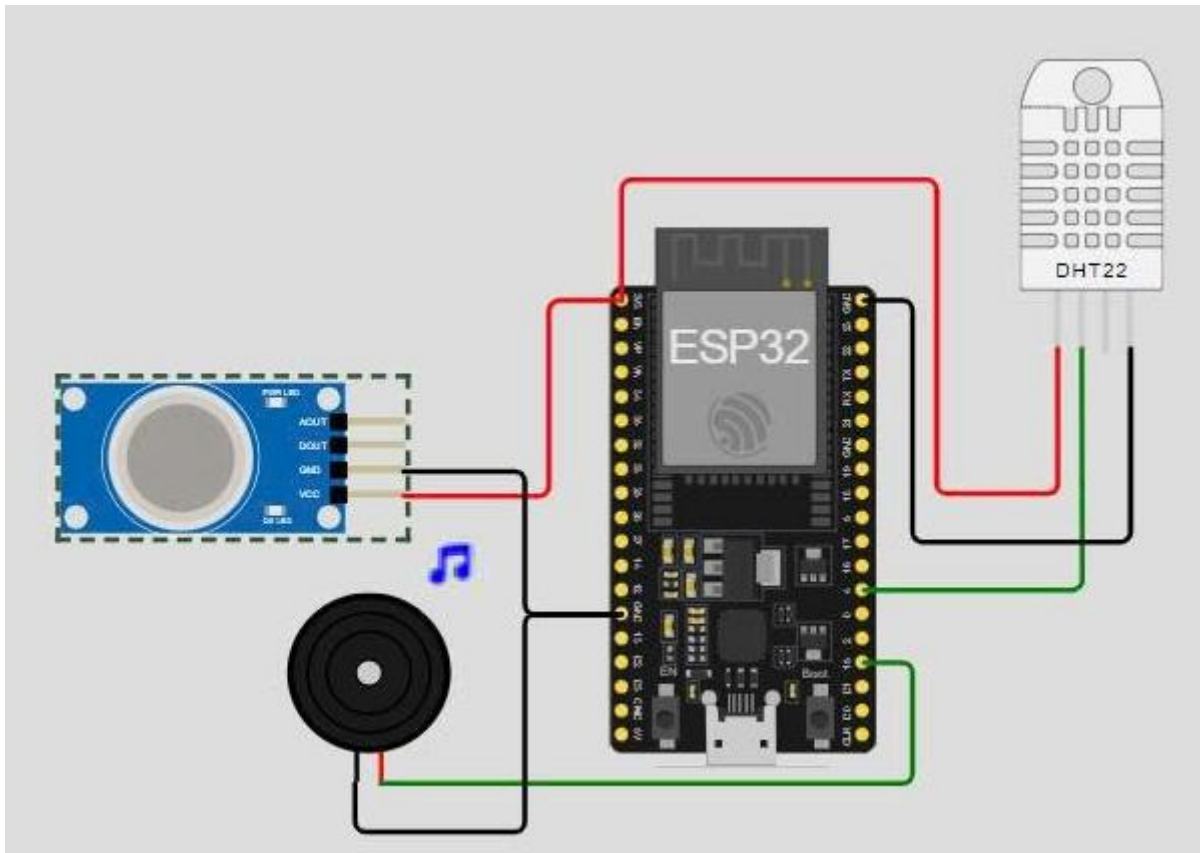


- Kết quả quan sát:
 - Buzzer tắt
 - Không có tin nhắn gửi qua Telegram
 - Serial Monitor hiển thị trạng thái bình thường

Trường hợp 2: Khói vượt ngưỡng, nhiệt độ bình thường

- Thiết lập mô phỏng:

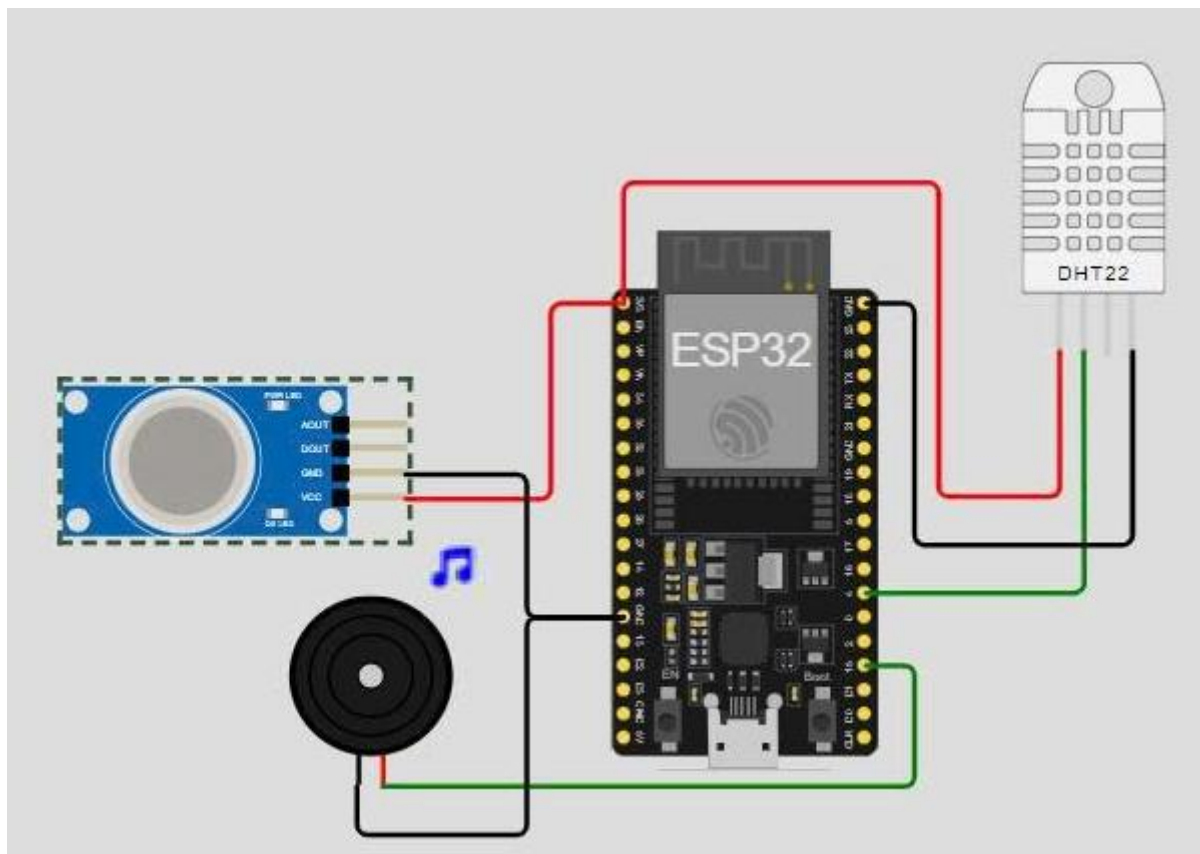
- MQ2 analog > 300 (khói > 6%)
- DHT22 temperature < 50°C



- Kết quả quan sát:
 - Buzzer kêu cảnh báo khói
 - Cảnh báo được gửi tới Telegram “Nồng độ khói vượt ngưỡng, nguy cơ cháy nổ cao.”
 - Serial Monitor hiển thị giá trị khói vượt ngưỡng

Trường hợp 3: Nhiệt độ vượt ngưỡng, khói bình thường

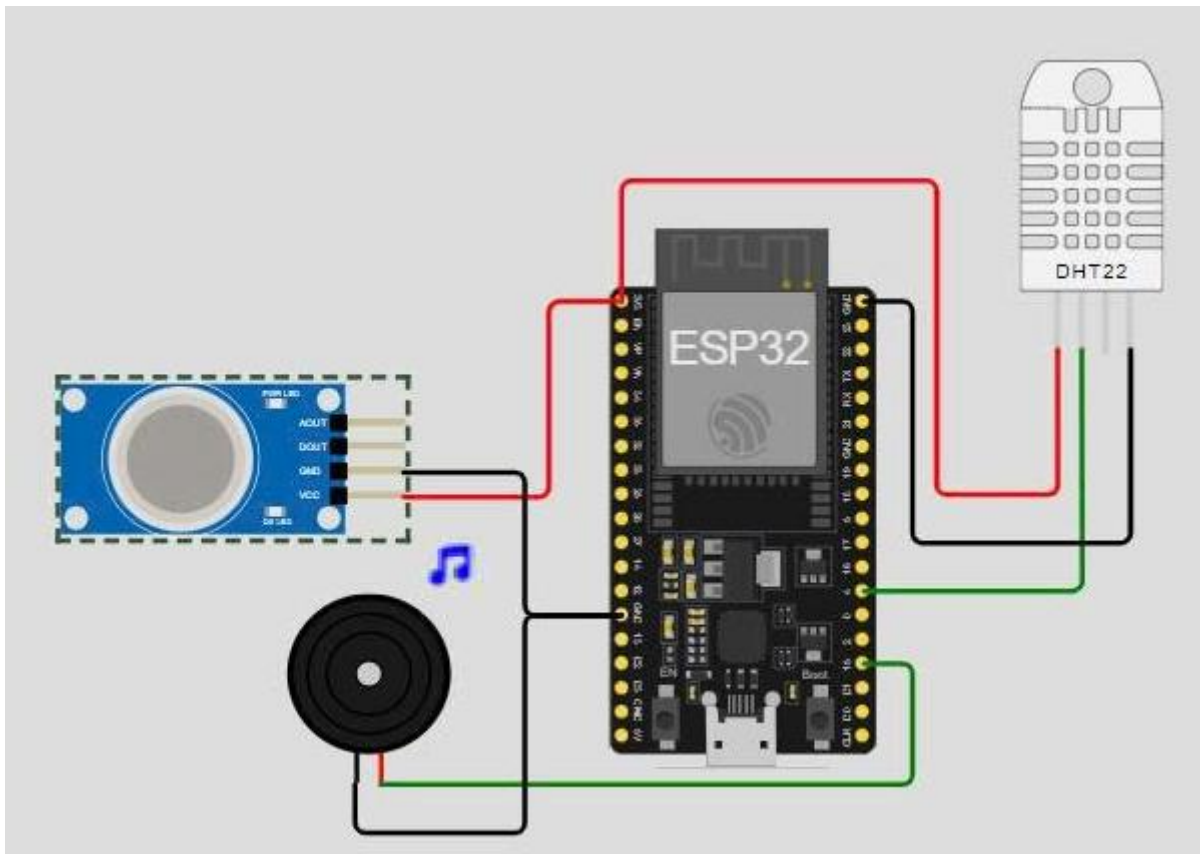
- Thiết lập mô phỏng:
 - MQ2 analog < 300 (khói < 6%)
 - DHT22 temperature > 50°C



- Kết quả quan sát:
 - Buzzer kêu cảnh báo nhiệt độ
 - Cảnh báo được gửi tới Telegram “*Nhiệt độ vượt ngưỡng, nguy cơ cháy nổ cao.*”
 - Serial Monitor hiển thị giá trị nhiệt độ vượt ngưỡng

Trường hợp 4: Cả khói và nhiệt độ đều vượt ngưỡng

- Thiết lập mô phỏng:
 - MQ2 analog > 300 (khói > 6%)
 - DHT22 temperature > 50°C



- Kết quả quan sát:
 - Buzzer kêu liên tục
 - Cảnh báo được gửi tới Telegram “*Nhiệt độ và nồng độ khói vượt ngưỡng, nguy cơ cháy nổ cao.*”
 - Serial Monitor hiển thị cả hai giá trị vượt

4.5. Tổng kết và đánh giá hệ thống

Sau khi mô phỏng hệ thống trên Wokwi, kết hợp ESP32, cảm biến MQ2, DHT22 và buzzer, có thể tổng kết và đánh giá như sau:

4.5.1. Hiệu quả hoạt động của hệ thống

Hệ thống hoạt động theo nguyên lý: ESP32 đọc dữ liệu từ cảm biến MQ2 (khói) và DHT22 (nhiệt độ) liên tục, so sánh với ngưỡng:

- Ngưỡng khói: MQ2 analog > 300 (~6% LEL / ~600 ppm khói)

- Ngưỡng nhiệt độ: $DHT22 > 50^{\circ}C$

Tùy vào giá trị cảm biến, hệ thống thực hiện:

1. Khói vượt ngưỡng: Buzzer kêu liên tục, tin nhắn “*Nồng độ khói vượt ngưỡng, nguy cơ cháy nổ cao.*” gửi đến Telegram.
2. Nhiệt độ vượt ngưỡng: Buzzer kêu, tin nhắn “*Nhiệt độ vượt ngưỡng, nguy cơ cháy nổ cao.*” gửi đến Telegram.
3. Khói và nhiệt độ cùng vượt ngưỡng: Buzzer kêu liên tục, tin nhắn gửi đồng thời “*Nhiệt độ và nồng độ khói vượt ngưỡng, nguy cơ cháy nổ cao.*”
4. Cả hai đều bình thường: Buzzer tắt, không gửi cảnh báo, hệ thống ở trạng thái giám sát bình thường.

Các giá trị và trạng thái được hiển thị trực tiếp trên Serial Monitor, giúp người dùng dễ theo dõi khi mô phỏng hoặc triển khai thực tế.

4.5.2. Ưu điểm của hệ thống

- Phản hồi nhanh: Cảnh báo ngay khi giá trị khói hoặc nhiệt độ vượt ngưỡng, giúp giảm nguy cơ cháy lan.
- Cảnh báo đa kênh: Kết hợp buzzer cục bộ và Telegram từ xa, người dùng nhận tin nhắn tức thì ngay cả khi không có mặt tại hiện trường.
- Dễ triển khai và mô phỏng: Hệ thống đơn giản, chi phí thấp, các linh kiện dễ tìm.
- Tính mở rộng cao: Có thể thêm nhiều cảm biến khác (CO, CO₂, O₂...) để nâng cao độ chính xác.
- Dễ bảo trì: ESP32 và các cảm biến đều có tài liệu phong phú, lập trình đơn giản, dễ nâng cấp firmware.

4.5.3. Hạn chế và nhược điểm

- Độ chính xác: MQ2 cần calibration định kỳ để đảm bảo ngưỡng cảnh báo phù hợp.
- Phụ thuộc mạng: Cảnh báo Telegram yêu cầu kết nối Wi-Fi và MQTT Broker; nếu mất mạng, cảnh báo từ xa không hoạt động.
- Giới hạn cảm biến: Hiện chỉ phát hiện khói và nhiệt độ, chưa phát hiện các khí độc khác hoặc nguồn cháy gián tiếp.
- Mô phỏng vs thực tế: Trên Wokwi mô phỏng dễ dàng, nhưng khi triển khai thực tế cần tính đến nhiều môi trường, vị trí lắp đặt và bảo vệ thiết bị khỏi môi trường khắc nghiệt.

KẾT LUẬN

Bài tiểu luận này đã trình bày quá trình nghiên cứu, thiết kế và mô phỏng một hệ thống báo cháy thông minh cơ bản, sử dụng vi điều khiển ESP32, kết hợp cảm biến khói MQ2 và cảm biến nhiệt độ DHT22. Mục tiêu chính là minh họa khả năng phát hiện đồng thời nguy cơ cháy dựa trên nồng độ khói và nhiệt độ, đồng thời truyền tải cảnh báo đến người dùng qua MQTT và Telegram.

Quá trình mô phỏng trên Wokwi đã thành công trong việc thể hiện logic hoạt động của hệ thống. Trong các trường hợp kiểm thử khác nhau, hệ thống phản ứng chính xác: kích hoạt Buzzer cảnh báo cục bộ và gửi thông báo đến Telegram khi nồng độ khói hoặc nhiệt độ vượt ngưỡng an toàn. Điều này chứng minh tiềm năng của việc kết hợp ESP32, các cảm biến phổ biến và giao thức MQTT trong việc xây dựng hệ thống báo cháy thông minh, mang lại cảnh báo sớm và từ xa.

Mặc dù mô hình hiện tại còn hạn chế, như sử dụng cảm biến đơn giản, ngưỡng cố định và thiếu các tính năng nâng cao, hệ thống vẫn cung cấp nền tảng vững chắc để hiểu nguyên lý hoạt động và khả năng ứng dụng của IoT trong phòng cháy chữa cháy.

Trong tương lai, hệ thống có thể được cải tiến bằng cách:

- Tích hợp thêm các loại cảm biến khác để tăng độ chính xác.
- Cải thiện thuật toán xử lý dữ liệu, giảm thiểu báo động giả.
- Bổ sung khả năng cấu hình ngưỡng từ xa và kết nối với các hệ thống PCCC chuyên nghiệp.
- Triển khai trên phần cứng thực tế với hiệu chuẩn cảm biến, đảm bảo kết nối mạng ổn định và nguồn điện an toàn, nâng cao độ tin cậy của toàn bộ hệ thống.

Với sự phát triển không ngừng của công nghệ IoT, các hệ thống báo cháy thông minh dựa trên ESP32 và MQTT hứa hẹn sẽ đóng vai trò quan trọng trong việc nâng cao hiệu quả phòng cháy chữa cháy, bảo vệ tính mạng và tài sản cộng đồng.

Tài liệu tham khảo

Tiếng Việt:

[1] IoTZone.vn. (n.d.). *Hướng dẫn DHT11, DHT22 ESP32 trên Arduino IDE*. Truy cập từ <https://www.iotzone.vn/esp32/huong-dan-dht11-dht22-esp32-tren-arduino-ide/>

[2] IoTZone.vn. (n.d.). *Hướng dẫn kết nối ESP32 với MQTT Broker*. Truy cập từ <https://www.iotzone.vn/esp32/mqtt-arduino-esp32/>

[3] Wiki Mắt Bão. (n.d.). *Hướng dẫn tạo bot và gửi thông báo Telegram*. Truy cập từ <https://wiki.matbao.net/kb/huong-dan-tao-bot-va-gui-thong-bao-telegram/>